

ATTACKS USING INSECURE CAPTCHA

Overview

This section introduces techniques for bypassing web application defenses (like CAPTCHAs) designed to stop automated attacks. It explores web application reconnaissance, tools for capturing and altering HTTP requests, and how these capabilities relate to adversary exploits such as remote code execution, Cross-Site Scripting (XSS), Man-in-the-Middle (MitM), data manipulation, and Denial-of-Service (DoS).

Objective

The goal is to analyze application code and logic to identify system vulnerabilities and achieve privilege escalation. Specifically, you will perform an account takeover on the System Under Test (SUT) by resetting/changing the administrator account's password and effectively locking out the legitimate user to lay the groundwork for potential lateral movement and deeper network access.

Strategy

The exercise tests my ability to exploit web vulnerabilities within a custom or extended Damn Vulnerable Web Application (DVWA) environment. The primary focus is exploiting a flawed state-management mechanism, specifically, a vulnerable HTML POST request utilizing hidden fields to bypass a validation step in the application's normal workflow.

Walkthrough

The exercise window displays “Change your password.”, and the CAPTCHA challenge, select the “View Source” button located on the bottom right of the client windows

The screenshot shows the DVWA interface. On the left is a navigation menu with options like Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA (highlighted), SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), and DVWA Security. The main content area is titled 'Vulnerability: Insecure CAPTCHA' and contains a 'Change your password:' form with fields for 'Current password:', 'New password:', and 'Confirm new password:'. Below the form is a CAPTCHA challenge with a checkbox 'I'm not a robot' and a 'Change' button. To the right of the form, a 'More Information' section lists links to Wikipedia, Google, and OWASP resources. A source code viewer window is open on the right, displaying the PHP code for the 'view_source.php' file. The code includes logic for checking the Anti-CSRF token, hiding the CAPTCHA form, and processing the password change request using md5 hashing.

```
<?php
if( isset( $_POST[ 'Change' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ] );

    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new = $_POST[ 'password_new' ];
    $pass_new = stripslashes( $pass_new );
    $pass_new = ((isset($_GLOBALS[ "__mysql_ston" ]) && is_object($
[MySQLConverterTool] Fix the mysql_escape_string() call! This code d
$pass_new = md5( $pass_new );
```

I then examined the HTTP calls being used by this web application. To do this, I use Wireshark. I started up Wireshark and ensure I was tracing the network the CTS server is located on. Once Wireshark is started and monitoring traffic, I input the new and confirmed password. And I do not provide the CAPTCHA challenge and select the “Change” button.

DVWA

Vulnerability: Insecure CAPTCHA

Change your password:

New password:

Confirm new password:

☐ I'm not a robot 

The CAPTCHA was incorrect. Please try again.

More Information

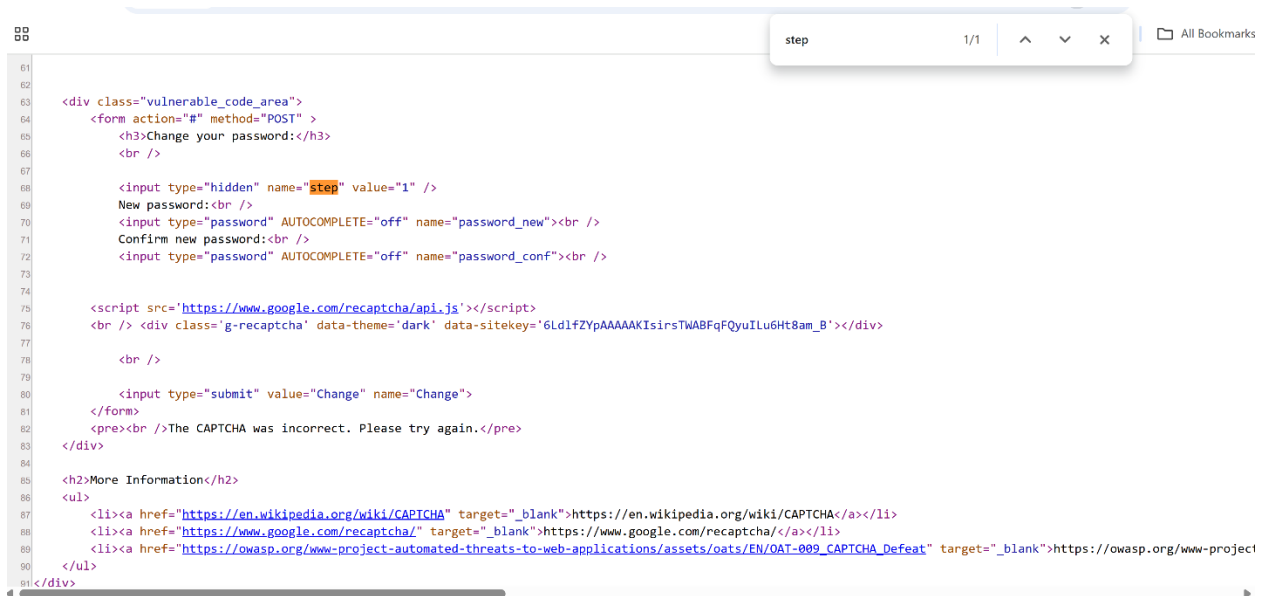
- <https://en.wikipedia.org/wiki/CAPTCHA>
- <https://www.google.com/recaptcha/>

Next, I stopped the Wireshark trace and filter on “http” traffic. And this is what I look at:

No.	Time	Source	Destination	Protocol	Length	Info
19	92.186839..	192.168.57.1	192.168.57.136	HTTP	850	POST /dvwa/vulnerabilities/captcha/ HTTP/1.1 (application/x-www-form-urlencoded)
46	92.405499..	192.168.57.136	192.168.57.1	HTTP	588	HTTP/1.1 200 OK (text/html)

Frame 19: Packet, 850 bytes on wire (6800 bits), 850 bytes captured (6800 bits) on interface \Device\NPF{...}	0000	00 0c 29 23 13 07 00 50 56 c0 00 08 08 00 45 00	..)###P V....E-
Ethernet II, Src: VMware_c0:00:08 (00:50:56:c0:00:08), Dst: VMware_23:13:07 (00:0c:29:23:13:07)	0010	03 44 ec f5 40 00 80 06 16 e4 c0 a8 39 01 c0 a8	.D. @... ..9...
Internet Protocol Version 4, Src: 192.168.57.1, Dst: 192.168.57.136	0020	39 88 2a 34 00 50 eb e2 da ad 37 34 71 a2 50 18	9:"4 P...74q P...
Transmission Control Protocol, Src Port: 10804, Dst Port: 80, Seq: 1, Ack: 796	0030	00 ff 42 60 00 00 50 4f 53 54 20 2f 64 76 77 61	..B"-PQ ST /dvwa
Hypertext Transfer Protocol	0040	2f 76 75 6c 6e 65 72 61 62 69 6c 69 74 69 65 73	/vulnerabilities
HTML Form URL Encoded: application/x-www-form-urlencoded	0050	2f 63 61 70 74 63 68 61 2f 20 48 54 54 50 2f 31	/captcha / HTTP/1
Form item: "step" = "1"	0060	2e 31 0d 0a 48 6f 73 74 3a 20 31 39 32 2e 31 36	.1..Host : 192.16
Form item: "password_new" = "123456"	0070	38 2e 35 37 2e 31 33 36 0d 0a 43 6f 6e 6e 65 63	8.57.136..Connec
Form item: "password_conf" = "123456"	0080	74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65	tion: ke ep-alive
Form item: "g-recaptcha-response" = ""	0090	0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74 68	..Conten t-length
Form item: "Change" = "Change"	00a0	3a 20 38 33 0d 0a 43 61 63 68 65 2d 43 6f 6e 74	: 83..Ca che-Cont
	00b0	72 6f 6e 3a 20 6d 61 78 2d 61 67 65 3d 30 0d 0a	rol: max -age=0
	00c0	55 70 67 72 61 64 65 2d 49 6e 73 65 63 75 72 65	Upgrade: Insecure
	00d0	2d 52 65 71 75 65 73 74 73 3a 20 31 0d 0a 43 6f	-Request s: 1..Co
	00e0	6e 74 65 6e 74 2d 54 79 70 65 3a 20 61 70 78 6c	ntent-ty pe: appl
	00f0	69 63 61 74 69 6f 6e 2f 78 2d 77 77 77 2d 66 6f	ication/ x-ww-fo
	0100	72 6d 2d 75 72 6c 65 6e 63 6f 64 65 64 0d 0a 55	rm-urlen coded-U
	0110	73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f 7a 69 6c	ser-Agen t: Mozil
	0120	6c 61 2f 35 2e 30 20 28 57 69 6e 64 6f 77 73 20	la/5.0 (Windows
	0130	4e 54 20 31 30 2e 30 3b 20 57 69 6e 36 3a 3b 20	NT 10.0; Windo
	0140	78 36 34 29 20 41 70 70 6c 65 57 65 62 4b 69 74	x64) App leWebKit
	0150	2f 35 33 37 2e 33 36 20 28 4b 48 54 4d 4c 2c 20	/537.36 (KHTML,
	0160	6c 69 6b 65 20 47 65 63 68 6f 29 20 43 68 72 6f	like Gec ko) Cheo
	0170	6d 65 2f 31 35 33 2e 30 20 2e 30 20 53 61 66	me/537.36.0 Saf
	0180	61 72 69 2f 35 33 37 2e 33 36 0d 0a 4f 72 69 67	ari/537.36..Orig
	0190	69 6e 3a 20 68 74 74 70 3a 2f 2f 31 39 32 2e 31	in: http ://192.1

I noticed the form data provided I determined the password aspects of the request, but I also saw a keyword of “step”. I needed to find out more about this field, so I decided to review the source of the page to perform my evaluation.



```
61
62
63 <div class="vulnerable_code_area">
64   <form action="#" method="POST" >
65     <h3>Change your password:</h3>
66     <br />
67     <input type="hidden" name="step" value="1" />
68     New password:<br />
69     <input type="password" AUTOCOMPLETE="off" name="password_new"><br />
70     Confirm new password:<br />
71     <input type="password" AUTOCOMPLETE="off" name="password_conf"><br />
72
73     <script src='https://www.google.com/recaptcha/api.js'></script>
74     <br /> <div class='g-recaptcha' data-theme='dark' data-sitekey='6LdlfZYpAAAAAKIsirsTWABFqQyuILu6Ht8am_B'></div>
75
76     <br />
77
78     <input type="submit" value="Change" name="Change">
79   </form>
80   <pre><br />The CAPTCHA was incorrect. Please try again.</pre>
81 </div>
82
83 <h2>More Information</h2>
84 <ul>
85   <li><a href="https://en.wikipedia.org/wiki/CAPTCHA" target="_blank">https://en.wikipedia.org/wiki/CAPTCHA</a></li>
86   <li><a href="https://www.google.com/recaptcha/" target="_blank">https://www.google.com/recaptcha/</a></li>
87   <li><a href="https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT-009_CAPTCHA_Defeat" target="_blank">https://owasp.org/www-project
88 </ul>
89 </div>
90
91
```

Because HTTP is a stateless protocol, what that means is that there is no state being shared between HTTP requests to order or associate them with each other. There are a couple of ways to perform this activity, the first (and very well-known) technique is to use cookies. The second technique is to use hidden fields to track state, which may be the case in this example. I assumed the state changes from 1, to 2, to 3, ... to completion. Having looked at the Wireshark trace, and now the source, I “guessed” about how the hidden field can be used to track state by using the hidden “step” input field, with the assumption that a state with the value of “1” performs CAPTCHA validation and transitions to state “2”, which may confirm the password matches, and a possible state of 3 performing the “change password” request.

I used the Web Developer. The most basic support for web development is provided in all common browsers such as Chrome and Firefox using the “Inspect” features of the applications. I looked within Chrome and the “Inspect” menu item.

The image shows a web browser displaying the DVWA (Damn Vulnerable Web Application) interface. The page is titled "Vulnerability: Insecure CAPTCHA". It features a form for changing a password, which includes a "New password:" field, a "Confirm new password:" field, and a "Change" button. A CAPTCHA image is displayed below the password fields, showing the text "I'm not a robot" and a reCAPTCHA logo. A message below the CAPTCHA states: "The CAPTCHA was incorrect. Please try again."

On the right side of the image, the browser's developer tools are open, showing the HTML source code. The code is structured as follows:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"
>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
</head>
<body class="home">
<div id="container">
<div id="header">
</div>
<div id="main_menu">
</div>
<div id="main_body">
<div class="body_padded">
<h1>Vulnerability: Insecure CAPTCHA</h1>
<div class="vulnerable_code_area">
<form action="#" method="POST">
<h3>Change your password:</h3>
<br>
<input type="hidden" name="step" value="1">
" New password:"
<br>
<input type="password" autocomplete="off" name="password_new"
fdprocessedid="3sp777">
<br>
" Confirm new password:"
</form>
</div>
</div>
</div>
</div>
</div>
</body>
</html>
```

The developer tools also show the CSS styles for the form elements, including margin, border, padding, font-size, and position.

In the example above, the element for the “password change” form has focus, and this reveals the actual code used to provide this functionality. In this example, I easily located the hidden “step” input type used for session control

To perform a test of my assumptions, I can double clicked on the value area of the “step” field, which opens an “edit” enabled textbox. In this textbox, change the value from “1” to “2”. I verified the changed value is preserved, set to 2. Once I verified this, I went to the “Change Password” section of the UI under “Vulnerability: Insecure CAPTCHA”.

The image shows the DVWA interface after the user has changed the value of the hidden “step” field from “1” to “2”. The page title remains “Vulnerability: Insecure CAPTCHA”. The form for changing a password is still present, but the CAPTCHA image is now a reCAPTCHA logo. A message below the CAPTCHA states: “The CAPTCHA was incorrect. Please try again.”

On the right side of the image, the browser's developer tools are open, showing the HTML source code. The code is structured as follows:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"
>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
</head>
<body class="home">
<div id="container">
<div id="header">
</div>
<div id="main_menu">
</div>
<div id="main_body">
<div class="body_padded">
<h1>Vulnerability: Insecure CAPTCHA</h1>
<div class="vulnerable_code_area">
<form action="#" method="POST">
<h3>Change your password:</h3>
<br>
<input type="hidden" name="step" value="2">
" New password:"
<br>
<input type="password" autocomplete="off" name="password_new"
fdprocessedid="3sp777">
<br>
" Confirm new password:"
</form>
</div>
</div>
</div>
</div>
</body>
</html>
```

The developer tools also show the CSS styles for the form elements, including margin, border, padding, font-size, and position.

A notification banner at the bottom of the browser window states: “You earned the Chrome DevTools User badge for the Google Developer Program! Create a profile to claim your badge.”

Provide the same input for the new password and confirmation input fields and select the “Change” button (note, don’t provide any response to the CAPTCHA challenge (and I don’t click the CAPTCHA Check Box), which I am trying to bypass. Because I changed the value from “1” to “2”, I got a confirmation “Password changed” as demonstrated below:

The screenshot displays the DVWA (Damn Vulnerable Web Application) interface. The top navigation bar includes links for Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA (highlighted), SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), DVWA Security, PHP Info, About, and Logout. The main content area is titled "Vulnerability: Insecure CAPTCHA" and shows a message "Password Changed." in red. Below this, the "More Information" section lists three links: <https://en.wikipedia.org/wiki/CAPTCHA>, <https://www.google.com/recaptcha/>, and https://owasp.org/www-project-automated-threats-to-web-applications/assets/oa009_CAPTCHA_Defeat. On the right side, a Google Password Manager overlay is visible, titled "Update password?". It shows the username "admin" and a masked password field. The overlay includes buttons for "Update password" and "No thanks", and a note about saved passwords on a device.